Mnemos Archive

Tus documentos, recordados.

Informe del mini-proyecto de la certificación
Microsoft DP-800: SQL AI Developer Associate

Álvaro García-Quismondo Lizana
Tajamar, Madrid · Octubre de 2026
Repositorio: github.com/alvarophck/mnemos-archive

1. Descripción

Un autónomo o una pequeña empresa acumula cientos de PDF: facturas emitidas y recibidas, contratos y presupuestos. Encontrar algo obliga a abrirlos uno a uno, y las preguntas útiles no se pueden hacer con una búsqueda por palabras: «¿cuánto le he facturado a Pedro este año?», «¿con cuánto preaviso puedo dejar la oficina?».

Mnemos Archive guarda cada documento en SQL Server 2025 con sus datos estructurados (tercero, fecha, base, IVA, total), con su texto troceado y convertido en vectores, y usa un modelo de lenguaje local para clasificar los documentos nuevos y responder preguntas citando siempre el documento y la página. Todo se ejecuta en el propio ordenador: sin nube, sin cuentas y sin claves de API.

El escenario de ejemplo es **Estudio Ámbar S.L.**, un estudio de diseño ficticio con 22 documentos y 11 clientes y proveedores. Todos los nombres, NIF e importes son inventados.

Nivel del proyecto: 3 (Avanzado). Diseño de base de datos, T-SQL, seguridad, optimización, embeddings, vector search y RAG con modelo local.

2. Arquitectura

Pieza	Papel
App web (.NET 9, Blazor)	Interfaz en el navegador, solo en 127.0.0.1:5080. Archivar, preguntar y ver documentos.
SQL Server 2025	Tablas, vectores `vector(1024)`, Row-Level Security, procedimientos. Llama al modelo con `CREATE EXTERNAL MODEL` y `AI_GENERATE_EMBEDDINGS`.
Caddy	Puente HTTPS con certificado local. SQL Server solo llama a modelos por HTTPS y Ollama habla HTTP.
Ollama	Ejecuta `bge-m3` (embeddings) y `qwen3:4b-instruct` (respuestas) en el ordenador.

Decisión clave: la lógica vive en la base de datos. La app es una interfaz sobre los mismos procedimientos que se pueden ejecutar desde SSMS. Se usa la variante *instruct* de qwen3 porque la variante por defecto razona en voz alta y tardaba hasta 200 segundos por respuesta.

3. Funcionalidades DP-800 implementadas

Bloque	Qué se hizo
Diseño de base de datos	6 tablas con PK, FK, UNIQUE, CHECK, DEFAULT y borrado en cascada
Vistas, funciones, procedimientos, triggers	4 vistas (una con PIVOT), 6 funciones, procedimientos relacionales y de IA, 3 triggers (auditoría, borrados, invalidar vectores)
T-SQL avanzado	CTE, funciones de ventana, LAG, subconsultas correlacionadas, JSON, `REGEXP_*`, fuzzy matching
Seguridad	Roles con GRANT/DENY/REVOKE, Row-Level Security, Dynamic Data Masking, usuarios sin login, auditoría
Optimización	Clustered Index Scan a Index Seek: 4.512 a 18 lecturas lógicas
Embeddings y vector search	Chunking, embeddings, `VECTOR_DISTANCE` coseno y ranking por documento
Búsqueda híbrida	Full-Text y vectores con Reciprocal Rank Fusion
RAG	Pregunta, vectores, contexto, modelo local y respuesta con cita
IA en la ingesta	El modelo clasifica y extrae campos con JSON Schema, validado en T-SQL
Desarrollo asistido por IA	Casos documentados en `docs/ia-asistida.md`
Pruebas	Pruebas en SQL (`tests/`) y 26 pruebas en .NET

4. Cómo responde: respuesta primero, fuente siempre

Preguntas sobre el contenido (RAG)

La pregunta se convierte en vector, se recuperan los fragmentos más cercanos y el modelo responde solo con ese contexto. La respuesta cita archivo y página, y el pasaje usado aparece resaltado. Si no lo encuentra, lo dice.



Pantalla Preguntar: respuesta, chip con archivo y página, pasaje resaltado.

Preguntas sobre cifras

El RAG solo ve unos pocos fragmentos, así que no sirve para contar ni sumar. Para esas preguntas el modelo solo *entiende* la pregunta y devuelve un JSON (tercero, fechas, operación). SQL Server calcula con una lista cerrada de operaciones. El modelo redacta, y antes de mostrar la respuesta se comprueba que cada número exista en lo calculado y que la respuesta contenga el resultado de SQL; si no, se muestra el texto fijo calculado.



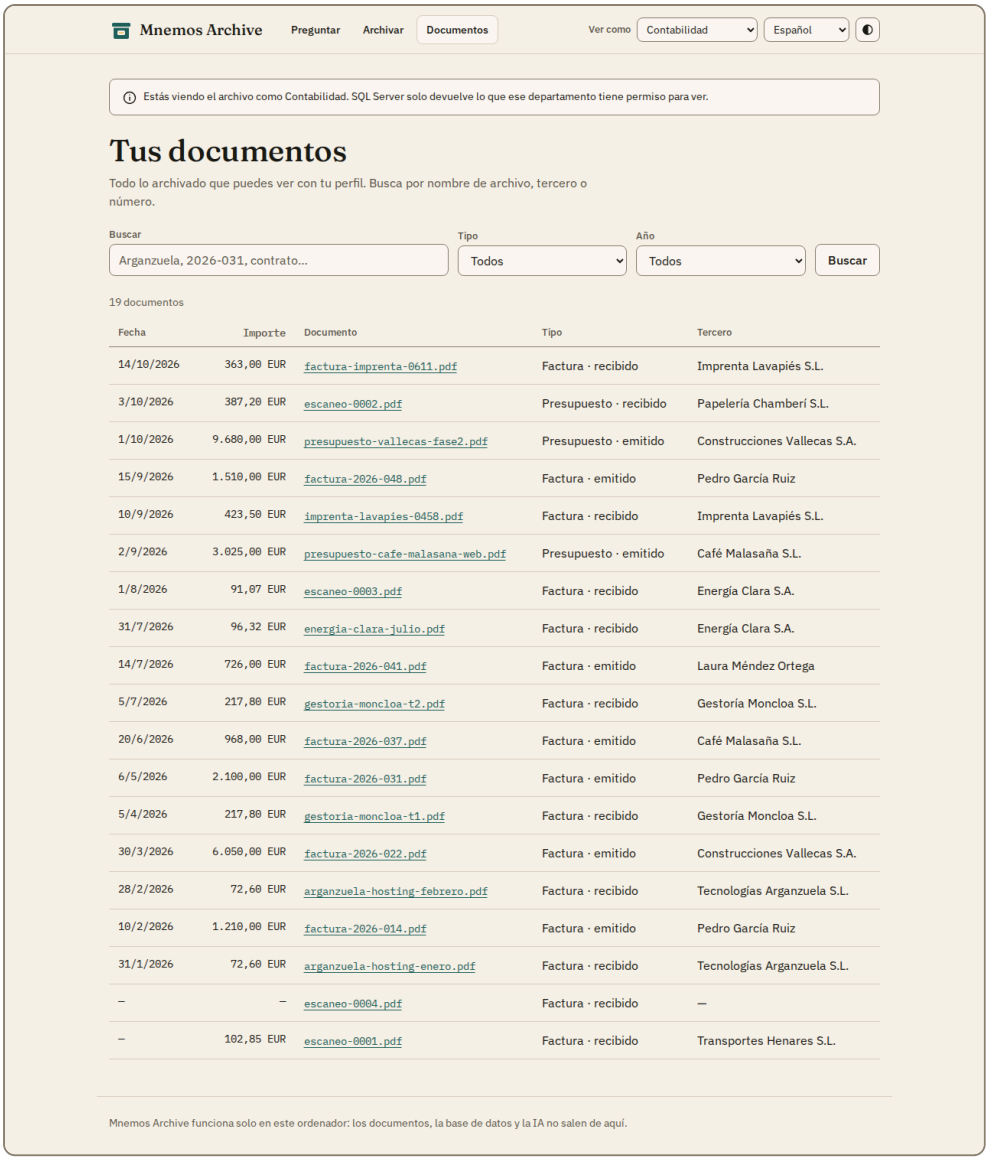
Pregunta de datos: la tabla muestra los documentos usados y se indica que las cifras se comprobaron.

Un fallo real: ante «el doble de lo que pagué», el modelo escribió «72,60 × 2 = 145,20» con cifras que sí existían en los datos pero con una operación falsa. La corrección fue quitarle la aritmética al modelo: ahora la hace T-SQL y una función comprueba que no aparezcan signos de operación inventados.

5. Seguridad

Row-Level Security añade un filtro por departamento a todas las consultas. Como el RAG y la búsqueda leen las tablas con RLS, cada usuario solo recibe respuestas de los documentos que puede ver. La misma consulta, con tres usuarios:

```
usr_admin     contrato 4 · factura 12 · presupuesto 2 · sin_clasificar 4    NIF 12345678Z
usr_contable  factura 12 · presupuesto 2 · sin_clasificar 4                 NIF *****678Z
usr_legal     contrato 4                                                    NIF -> permiso denegado
```



La app vista como Contabilidad: no aparecen los contratos.

Borrar un documento borra en cascada sus campos, fragmentos y vectores. La auditoría registra que se borró, pero no el contenido (derecho de supresión del RGPD en una sola llamada).

6. Optimización

Consulta: documentos de un tercero en un rango de fechas, con 200.000 documentos.

	Plan	Lecturas lógicas	Tiempo
Antes	Clustered Index Scan + Sort	4.512	28 ms
Después	Index Seek (sin Sort)	18	2 ms

El primer índice propuesto no se usaba: Row-Level Security añade un filtro por `departamento` y esa columna no estaba en el índice. Añadirla al `INCLUDE` convirtió el Scan en Seek. Cifras medidas en el entorno de pruebas; la salida real está en `evidencias/database_11_optimization.txt`.

7. Desarrollo asistido por IA

El proyecto se desarrolló con Claude (Anthropic) como asistente. En `docs/ia-asistida.md` está cada caso: el prompt, la respuesta, qué se aceptó, qué se corrigió y cómo se verificó. Problemas reales del código generado que aparecieron al ejecutarlo:

Problema	Solución
Tenía SQL Server 2022 (16.0) y no 2025	`check-env.ps1` lo detectó; se instaló una instancia `SQL2025`
SQL Server solo llama a modelos por HTTPS	Caddy delante de Ollama con certificado local
Índice filtrado: error 1934	`SET QUOTED_IDENTIFIER ON` y `sqlcmd -I`
Funciones de ventana anidadas (error 4109)	Calcular los totales en un CTE y ordenar después
`REGEXP_MATCHES` rechaza `nvarchar(max)` (error 19304)	Convertir a `nvarchar(4000)`
Una función escalar con REGEXP tumbaba el servicio	Reescrita como función en línea
`qwen3:4b` razona en voz alta (error 31629 por timeout)	Usar `qwen3:4b-instruct`
Fecha vacía convertida a 1900-01-01	`NULLIF` antes de `TRY_CONVERT`
El modelo inventaba cuentas («el doble», «prorrateado»)	Operaciones calculadas en T-SQL y verificadas

8. Cómo ejecutarlo

```
.\scripts\check-env.ps1      # comprueba SQL Server, Ollama, modelos y Caddy
.\scripts\instalar.ps1       # crea la base de datos, ejecuta todo y guarda
evidencias.\scripts\abrir-app.ps1      # abre la app en http://127.0.0.1:5080
```

Los requisitos están en `REQUISITOS.md` y los pasos en `GUIA-DE-USO.md` y en el `README.md`. La salida real de cada script queda en la carpeta `evidencias/`.

9. Conclusiones

- SQL Server 2025 puede hacer todo el ciclo de IA sin salir de T-SQL: trocear, vectorizar, buscar y llamar a un modelo local.
- Un RAG es tan bueno como su recuperación: lo que no llega al contexto, el modelo no lo puede usar. Por eso cada fragmento se vectoriza con una cabecera (tipo, tercero, archivo).
- Las cifras no se le piden al modelo: se calculan en SQL y se comprueban.
- La seguridad se hereda: RAG y búsqueda leen las tablas con RLS.

Limitaciones y mejoras posibles

- Los PDF escaneados (fotos) no se archivan: no hay OCR en esta versión.
- Si el documento nombra a una empresa que no existe, se archiva «sin enlazar»; falta crear el tercero automáticamente.
- Índice vectorial DiskANN cuando el archivo crezca, y fragmentación por párrafos o cláusulas.
- Instalador con icono para usuarios no técnicos.

Capturas tomadas en el entorno de pruebas con los datos ficticios de Estudio Ámbar S.L. Código bajo licencia MIT.